

Service discovery

Service Discovery

概念：Service Discovery 是指在计算机网络中自动定位服务的过程。在分布式系统中，服务的位置和可用性可能会动态变化，因此需要一种机制来自动发现和定位服务。

具体来说，Service Discovery 通常涉及一个服务注册表（Service Registry），该注册表记录了各个服务的位置和状态信息。当服务启动时，它们会向服务注册表注册自己的位置和状态；当服务停止或出现故障时，它们会从服务注册表中注销自己。这样，其他服务就可以通过查询服务注册表来找到可用的服务实例。

Service Discovery Patterns(实现方式)

1.客户端发现模式（Client-Side Discovery）：

- **实现方式：**客户端直接从服务注册表（Service Registry）查询目标服务的位置信息。然后客户端选择一个服务实例并直接与其通信。
- **优点：**简单、直接，没有额外的中间层。
- **缺点：**客户端需要实现服务发现逻辑，并且需要了解服务注册表的位置和 API。不同的编程语言，它们中的每一个都

需要实现客户端服务器发现才能相互通信，服务和注册表之间存在更多耦合。

- **应用场景:** 适用于客户端能够轻松访问注册表并具有一定的复杂性处理服务发现逻辑的场景。
- **示例:** Netflix Eureka 与 Ribbon 的结合。

2. 服务器端发现模式 (Server-Side Discovery) :

- **实现方式:** 客户端通过一个负载均衡器或 API 网关发送请求，负载均衡器或 API 网关从注册表查询服务实例，并将请求转发到合适的服务实例。
- **优点:** 客户端简单，不需要实现服务发现逻辑；集中管理服务发现逻辑。它与语言无关。不需要为您拥有的每种类型的微服务进行特定的实现。而且，由于它是检查您的服务的外部应用程序，即使您的服务挂起或没有响应，该第三方应用程序也可以检查状态并为您更新注册表。
- **缺点:** 增加了负载均衡器或 API 网关的复杂性和负载。
- **应用场景:** 适用于需要将服务发现逻辑从客户端中解耦的场景，特别是在大型系统中。
- **示例:** AWS Elastic Load Balancer (ELB) 与 Consul 的结合。

3. 自动注册 (Self-Registration)

实现方式

1. **服务启动时注册**: 服务实例在启动时, 通过 **API** 将自己的位置信息 (如 **IP 地址**、**端口**、**服务名称**等) 注册到服务注册表中。
2. **定期心跳报告**: 服务实例定期向服务注册表发送心跳报告, 以维持注册状态并表明自己是健康的。
3. **服务关闭时注销**: 服务实例在正常关闭时, 主动向服务注册表发送注销请求, 移除自己的注册信息。

优点:

- **简单直接**: 服务实例自己处理注册和注销逻辑, 无需外部工具的干预。
- **即时更新**: 服务实例的注册状态可以及时反映其启动和关闭状态。

缺点

- **增加复杂性**: 每个服务实例需要实现注册和注销逻辑, 这增加了代码的复杂性。

- **故障处理:** 如果服务实例异常退出,可能不会及时注销,导致服务注册表中存在“僵尸”实例。

应用场景:

适用于那些具有内置服务发现客户端库的服务框架,比如 Spring Cloud 的 Eureka 客户端,或者那些需要较高自主性和灵活性的场景。

4.第三方注册 (Third-Party Registration)

实现方式:

1. **外部服务监控:** 一个外部服务(如部署工具、容器编排系统或代理)监控服务实例的启动和关闭。
2. **自动注册和注销:** 外部服务根据监控结果,自动将服务实例的信息注册到服务注册表中或从中移除。

优点:

- **集中管理:** 服务注册逻辑集中在外部服务中,简化了服务实例的实现。
- **更好的故障处理:** 外部服务可以更好地处理服务实例的异常退出,及时更新服务注册表。

缺点：

- **依赖外部服务：**服务注册依赖于外部服务的可靠性和功能。
- **延迟问题：**监控和注册可能存在一定的延迟，影响服务实例的实时性。

应用场景：

适用于那些使用容器编排系统（如 **Kubernetes**）、需要集中管理服务注册逻辑的大型系统。